

# Flow-Matching Phantom Generation & Interpolation

Project summary — diffusion / flow-matching course project

## What this project is

The goal is to train a generative model over a dataset of 3D breast-phantom voxel scans, then use that model to interpolate between two real phantoms — not by blending their voxels directly, but by inverting each phantom into the model’s underlying noise representation, interpolating there, and generating a new phantom back out. The idea is that a smooth path through the model’s internal representation should correspond to a smooth, anatomically plausible morph between the two real examples, rather than a naive crossfade.

## The data

Each phantom is a 3D voxel volume ( $K=26$  depth slices  $\times H=128 \times W=128$ ), where every voxel is one of 4 classes: background, insert, pillar, or lump (the class that actually matters medically, and the rarest one). Raw voxel values  $\{0, 127, 200, 255\}$  are remapped to class indices  $\{0, 1, 2, 3\}$ . After removing double/triple-lump phantoms (out of scope for this single-lump framing), 123 files across 40 case IDs remain, expanded to about 984 volumes via  $8\times$  in-plane rotation augmentation on the training split.

## Why a generative model at all

A direct voxel-space crossfade between two phantoms would just produce a physically meaningless blur — two lumps fading in and out of each other. Training a generative model first means it learns a continuous, structured way to turn noise into valid-looking anatomy. If that structure is smooth enough, moving smoothly through the model’s noise representation should also move smoothly through valid anatomy — turning an interpolation problem in a messy, discrete voxel space into an interpolation problem in a continuous space the model was actually built to understand.

## The flow-matching process, from first principles

### 1. What we’re trying to learn

We have a data distribution we can sample from (real phantoms, one-hot encoded over the 4 classes so each voxel becomes a length-4 vector) but have no formula for. Call a sample from it  $x_1$ . We also have a distribution we understand completely and can sample trivially: standard Gaussian noise,  $x_0 \sim \mathcal{N}(0, I)$ , same shape as  $x_1$ . The goal is to learn a transformation that turns noise into data — concretely, a rule for continuously moving a point from  $x_0$  at “time”  $t = 0$  to  $x_1$  at  $t = 1$ .

## 2. Pick a path between them

The simplest possible path between two points is a straight line. For one paired sample  $(x_0, x_1)$ , define the point along that line at time  $t \in [0, 1]$  as

$$x_t = (1 - t)x_0 + tx_1. \quad (1)$$

At  $t = 0$  this is exactly the noise; at  $t = 1$  it's exactly the real phantom; in between it's a linear blend of the two. Differentiating Eq. (1) with respect to  $t$  gives the velocity a point moving along this line has at every instant — and because the path is a straight line, this velocity is simply constant along the whole path:

$$\frac{dx_t}{dt} = x_1 - x_0 =: v^*. \quad (2)$$

$v^*$  is the target (or conditional) velocity for this specific pair: the same number for every  $t$  along this one path, but a different number for every different pair  $(x_0, x_1)$  we could have drawn.

## 3. Learn a single velocity field that works for every pair

We can't store a separate velocity for every possible  $(x_0, x_1)$  pair — there are infinitely many. Instead we train a neural network  $v_\theta(x, t)$  that takes any point  $x$  and time  $t$  and predicts what the velocity should be there, trained by regressing onto  $v^*$  at randomly sampled points along randomly sampled paths:

$$\mathcal{L}(\theta) = \mathbb{E}_{t \sim U[0,1], x_0 \sim \mathcal{N}(0, I), x_1 \sim p_{\text{data}}} \left[ \|v_\theta(x_t, t) - (x_1 - x_0)\|^2 \right]. \quad (3)$$

At every training step: sample a real phantom  $x_1$ , sample fresh noise  $x_0$ , sample a random  $t$ , form  $x_t$  by Eq. (1), run the network on  $(x_t, t)$ , and push its output toward  $x_1 - x_0$ . Over many such triples, the network is forced to learn a single, shared velocity field consistent with every straight-line path it was shown, rather than memorizing any one of them.

# Sampling, inversion, and the interpolation idea

## 4. Generating a new sample: integrate the field forward

Once  $v_\theta$  is trained, generating a new phantom means starting at pure noise  $x_0 \sim \mathcal{N}(0, I)$  at  $t = 0$  and numerically walking forward to  $t = 1$ . Euler's method splits  $[0, 1]$  into  $n$  steps of size  $\Delta t = 1/n$  and repeats

$$x_{t+\Delta t} = x_t + v_\theta(x_t, t) \Delta t. \quad (4)$$

After  $n$  steps this produces a generated  $\hat{x}_1$  — a continuous, 4-channel volume. Taking the argmax over the 4 channels at every voxel turns it back into a discrete label volume that can be rendered. This is what `ode.py`'s `sample()` does; `n_steps` trades accuracy for compute.

## 5. Inverting a real phantom: integrate the field backward

The learned field also lets us go the other way. Given a real phantom  $x_1$ , integrating the same equation backward from  $t = 1$  to  $t = 0$ ,

$$x_{t-\Delta t} = x_t - v_\theta(x_t, t) \Delta t, \quad (5)$$

recovers an approximate noise pre-image  $\hat{x}_0$  for it. If the network has learned the field well, integrating forward from  $\hat{x}_0$  should reconstruct something very close to the original  $x_1$ . We verified this directly with a round-trip test (Section 6) and measured 96–99% voxel agreement at 50 integration steps.

## 6. The actual interpolation idea

To morph between two real phantoms A and B: invert both to get  $\hat{x}_0^A$  and  $\hat{x}_0^B$ , interpolate between these two noise representations at mixing fraction  $\alpha \in [0, 1]$ , then integrate the interpolated point forward to  $t = 1$ . A plain linear blend of two Gaussian vectors isn’t right, though (Section 4.3 makes this precise): spherical interpolation (slerp) moves along the great-circle arc between the two vectors instead, preserving their norm the whole way,

$$\text{slerp}(a, b, \alpha) = \frac{\sin((1 - \alpha)\theta)}{\sin \theta} a + \frac{\sin(\alpha\theta)}{\sin \theta} b, \quad \theta = \arccos\left(\frac{a \cdot b}{\|a\| \|b\|}\right). \quad (6)$$

The full pipeline per  $\alpha$  step: invert A and B to noise, slerp between them at that  $\alpha$ , integrate forward through  $v_\theta$ , argmax to a label volume, render. The noise-interpolation variant skips inversion and slerps between two directly-sampled Gaussian points instead.

## Dimensionality and distributions, in detail

This section works through exactly what shape and distribution every object in the pipeline has, because the interpolation failure we diagnose in Section 6 turns out to be a direct consequence of these numbers, not of a bug.

### 4.1 Exact shapes

Every one of  $x_0$ ,  $x_1$ ,  $x_t$ ,  $v_\theta(x_t, t)$ , and  $v^*$  has the identical shape: 4 channels  $\times$  26 depth slices  $\times$   $128 \times 128$ . Flattened into a single vector, that is a space of dimension

$$d = 4 \times 26 \times 128 \times 128 = 1,703,936. \quad (7)$$

Every noise sample, every real phantom, every intermediate  $x_t$ , and the noise pre-image  $\hat{x}_0$  from inversion are all points in this same  $d \approx 1.7$  million dimensional space. There is no bottleneck anywhere in this architecture — see 4.4.

### 4.2 The distributions at each stage

$x_0 \sim \mathcal{N}(0, I_d)$  by construction: every coordinate is an independent standard Gaussian.  $x_1$  is not continuous at all: each voxel is constrained to be one of the 4 one-hot corners of a simplex, so  $x_1$  lives on a discrete, highly constrained subset of  $\mathbb{R}^d$  (only a tiny sliver of which corresponds to real anatomy). For a *fixed* real phantom  $x_1$ , Eq. (1) is an affine function of the Gaussian  $x_0$ , so  $x_t$  conditioned on  $x_1$  is itself Gaussian:

$$x_t \mid x_1 \sim \mathcal{N}(tx_1, (1 - t)^2 I_d). \quad (8)$$

This collapses to  $\mathcal{N}(0, I_d)$  at  $t = 0$  and to a point mass at  $x_1$  at  $t = 1$ , exactly as expected. Averaging Eq. (8) over the whole training distribution of real phantoms gives the true *marginal* distribution the network sees  $x_t$  drawn from at every  $t$ :

$$p_t(x) = \mathbb{E}_{x_1 \sim p_{\text{data}}} \left[ \mathcal{N}(x; tx_1, (1 - t)^2 I_d) \right], \quad (9)$$

a continuous mixture of Gaussians with one mixture component centered at  $t \cdot x_1$  for every real phantom in the data.

### 4.3 High-dimensional geometry: why slerp and not a linear blend

**Intuition first.** A noise sample is  $d \approx 1.7$  million independent random numbers, one per voxel-channel. No individual number is predictable. But its *length* — distance from the origin, computed the ordinary way (square every coordinate, add them all up, take the square root) — is a sum of 1.7 million independent random pieces. Individually each piece is random, but adding up millions of independent random things makes the total extremely predictable: it is the same reason a casino always turns a profit even though any single spin of the wheel is random. So even though no single coordinate of a noise vector can be predicted, its overall length can be, almost exactly. That length is what the formulas below pin down as  $\approx 1305$ .

Now picture two arrows of equal length in ordinary 3D space, pointing in different directions. Add them tip-to-tail and take the midpoint: that midpoint arrow is *shorter* than either original, because the two arrows form a narrow “V” and the midpoint sits inside it, closer to the vertex. Only if the two arrows point in exactly the same direction does averaging preserve the length. In a space with 1.7 million dimensions, two random directions are essentially always close to *perpendicular* to each other, and averaging two equal-length, perpendicular arrows shrinks the result by a fixed factor of  $1/\sqrt{2} \approx 0.71$ . That is the entire reason a linear blend of two noise vectors comes out too short — it is just this “V” shrinkage, happening in a space too big to visualize.

Spherical interpolation (slerp) avoids the shrinkage differently: instead of cutting straight through the middle of the “V” (which is what shrinks the length), it walks along the surface of the sphere connecting the two points — like flying from New York to London along the Earth’s curved surface instead of drilling straight through the planet’s core. Every point on that curved flight path is still exactly the same distance from the Earth’s center as New York and London are. Slerp’s output is always the correct length, for every mixing fraction, no matter what.

**Now the same argument with the numbers.** For  $X \sim \mathcal{N}(0, I_d)$ ,  $\|X\|^2$  follows a chi-squared distribution with  $d$  degrees of freedom: mean  $d$ , variance  $2d$ . Using  $d = 1,703,936$  from Eq. (7),

$$\mathbb{E}[\|X\|^2] = d \approx 1,703,936, \quad \mathbb{E}[\|X\|] \approx \sqrt{d} \approx 1305.4. \quad (10)$$

By the delta method,  $\text{Var}(\|X\|) \approx \text{Var}(\|X\|^2)/(4d) = 2d/4d = 1/2$ , i.e.  $\text{SD}(\|X\|) \approx 0.71$  — a constant, *not growing with  $d$* . The relative fluctuation of the norm is therefore about  $0.71/1305.4 \approx 0.05\%$ : essentially every Gaussian sample lands within a hair’s width of the same radius, on a thin “shell” around the origin (the “casino” argument above, made precise). This is the concentration-of-measure phenomenon.

Now take two independent samples  $a, b \sim \mathcal{N}(0, I_d)$  and average them,  $m = (a + b)/2$ . Since  $a, b$  are independent,  $m \sim \mathcal{N}(0, I_d/2)$ , so by the same formula

$$\mathbb{E}[\|m\|] \approx \sqrt{d/2} \approx 923.1, \quad (11)$$

about  $70.7\% \approx 1/\sqrt{2}$  of the typical shell radius 1305.4 — exactly the “V” shrinkage described above, now with a number attached. The midpoint of two random Gaussian samples reliably lands at a norm around 923, nowhere near the shell (radius  $\approx 1305$ ) that Gaussian noise — and every noise pre-image the model was actually trained near — lives on. A linear blend is handing the network an input from a part of space it essentially never sees. Spherical interpolation avoids this:

for  $\|a\| = \|b\| = r$ , Eq. (6) gives  $\|\text{slerp}(a, b, \alpha)\| = r$  exactly, for every  $\alpha$  — the interpolated point stays on the same shell as the two endpoints by construction.

**What slerp does and doesn’t guarantee.** It is worth being precise about what this buys us, because it is easy to overstate. Slerp guarantees that  $z_\alpha$  sits at the correct *angular* position between  $\hat{x}_0^A$  and  $\hat{x}_0^B$  on the sphere — at  $\alpha = 0.5$  it is, by literal construction, the exact geographic midpoint of the flight path between them. That much is not an assumption; it follows directly from the formula. What it does *not* guarantee is that this midpoint *decodes* to an anatomy that is “in between” phantom A and phantom B. The network is a smooth, continuous function, so it changes gradually as you slide along the arc from A to B — but gradual is not the same as meaningful. Continuity only forbids sudden jumps; it does not forbid the decoded shape gradually tearing into a fragmented mess and back. The network was only ever taught the translation from noise-sphere position to anatomy *at*  $\hat{x}_0^A$  and *at*  $\hat{x}_0^B$  (Section 4.5): nobody ever told it what the geographic midpoint of that specific flight path should mean. Slerp guarantees we ask the network a fair, correctly-scaled question; it does not guarantee the network knows the answer. That gap — between asking a fair question and getting a meaningful answer — is precisely the failure diagnosed in Section 6, and precisely what  $t_{\text{mid}}$  (Section 7) targets: moving the question to a place closer to territory the network actually studied.

#### 4.4 Compression: what flow matching does *not* do

A VAE compresses data through a bottleneck — typically a few hundred to a few thousand numbers — and an explicit KL-divergence term during training forces that bottleneck’s distribution toward a simple, smooth prior. That explicit pressure to compress is exactly what makes VAE latent spaces organize into something that interpolates nicely: with no room to memorize per-example detail, the model is forced to generalize.

Flow matching does not compress at all. From Eq. (7),  $x_0$  lives in the *same*  $d \approx 1.7$  million dimensional space as the data — there is no bottleneck, and nothing in the loss (Eq. (3)) ever pushes the noise space to be smoothly *organized* as a whole. The loss only ever asks the network to get one straight-line path right at a time. That is a deliberate trade-off (no compression means no lost detail, which is why diffusion/flow models produce sharper samples than typical VAEs), but it is also exactly why individual inversion is excellent while interpolation between two different phantoms’ noise pre-images is not: nothing in training ever touches the region *between* two different paths.

#### 4.5 How sparse is the training data in this space?

The training set has on the order of  $10^3$  augmented volumes (984, from  $8\times$  rotation augmentation of the 40 case IDs). Those are  $10^3$  points scattered across a space of dimension  $d \approx 1.7 \times 10^6$ . There is no meaningful sense in which  $10^3$  points can densely cover a 1.7-million-dimensional space; the network only ever gets direct signal along the specific handful of straight lines it was shown, and has to extrapolate everywhere else. This is the quantitative version of the “uneven coverage” explanation in Section 6.

#### 4.6 Why the rendered noise pre-image ( $z$ ) looks the wrong size

Real phantom voxels are drastically imbalanced: measured directly from the data,

$$\text{background} = 50.8\%, \quad \text{insert} = 48.0\%, \quad \text{pillar} = 0.32\%, \quad \text{lump} = 0.94\%. \quad (12)$$

That imbalance is a property of real anatomy, not of the representation. A noise pre-image  $\hat{x}_0$ , however, is close to i.i.d. Gaussian per channel per voxel; by symmetry, each of the 4 channels wins the per-voxel argmax with probability  $1/4 = 25\%$ . So when we render  $\hat{x}_0$  using the same argmax-then-mesh-extract pipeline used for real data, the lump channel — which wins about 0.94% of voxels in real anatomy — suddenly wins about 25% of all voxels, roughly a  $26\times$  increase in footprint. That is exactly why the “z” panel in Figure 1 shows a lump region far larger than any real phantom’s, and why it extends all the way to the edges of the volume: real data always has background surrounding the anatomy acting as a boundary, and noise has no such boundary.

## Loss functions: plain MSE, then class-weighted MSE

### The starting point: unweighted per-voxel MSE

The loss in Eq. (3) is a per-voxel, per-channel squared error averaged uniformly over every voxel and all 4 channels. It has no notion of “class”: a lump voxel is worth exactly as much as a background voxel in the average.

### Why that’s a problem here

From Eq. (12), background and insert together make up almost the entire voxel count. A model can get the loss almost perfectly low just by nailing those two classes, while being essentially wrong about the lump with barely any effect on the reported loss. This matched what we observed directly: the overall phantom shape was learned solidly and consistently, while the lump was inconsistent — present, shrunk, or missing across training checkpoints and across repeated samples from the same fixed noise seed.

### The fix that was tried: class-weighted MSE

Multiply the squared error at each voxel by a weight depending on its true class, before averaging:

$$\mathcal{L}_{\text{weighted}}(\theta) = \mathbb{E} \left[ \frac{1}{C \cdot D} \sum_{c=1}^C w_c \sum_{\text{voxels}} (v_\theta - v^*)_{c,\text{voxel}}^2 \right], \quad (13)$$

with  $w_c$  set to the square root of inverse class frequency, normalized so background= 1:

$$w = (1.0, 1.03, 12.57, 7.36) \quad \text{for (background, insert, pillar, lump)}. \quad (14)$$

Square root rather than raw inverse frequency deliberately: raw inverse frequency would give pillar  $\approx 158\times$  the weight of background, risking letting a handful of rare voxels dominate the gradient and destabilize training.

### What actually happened

Plain unconditional generation looked about the same, maybe marginally more consistent about including a lump. Real-phantom interpolation got noticeably *worse*: instead of a clean shape with a duplicated lump (the old failure mode), the interpolated shape itself started tearing and fragmenting. Section 6 works out why.

## Diagnostics: visualizing and ruling things out

### Visualizing the round trip directly

Beyond the voxel-agreement number, we built a script (`flow_model/visualize_inversion.py`) that inverts a real, randomly-rotated (augmented) validation phantom to its noise pre-image and re-generates it, logging a before  $\rightarrow z \rightarrow$  after figure to Weights & Biases. Figure 1 shows one example (checkpoint: 1000-epoch, class-weighted loss). The reconstruction (“after”) closely matches the original (“before”), at 92.7% voxel agreement — consistent with the round-trip numbers measured earlier. The middle panel (“ $z$ ”) is explained in Section 4.6: it is not a bug that it looks large and boxy, that is the expected appearance of `argmax`’d near-Gaussian noise given the real class imbalance.

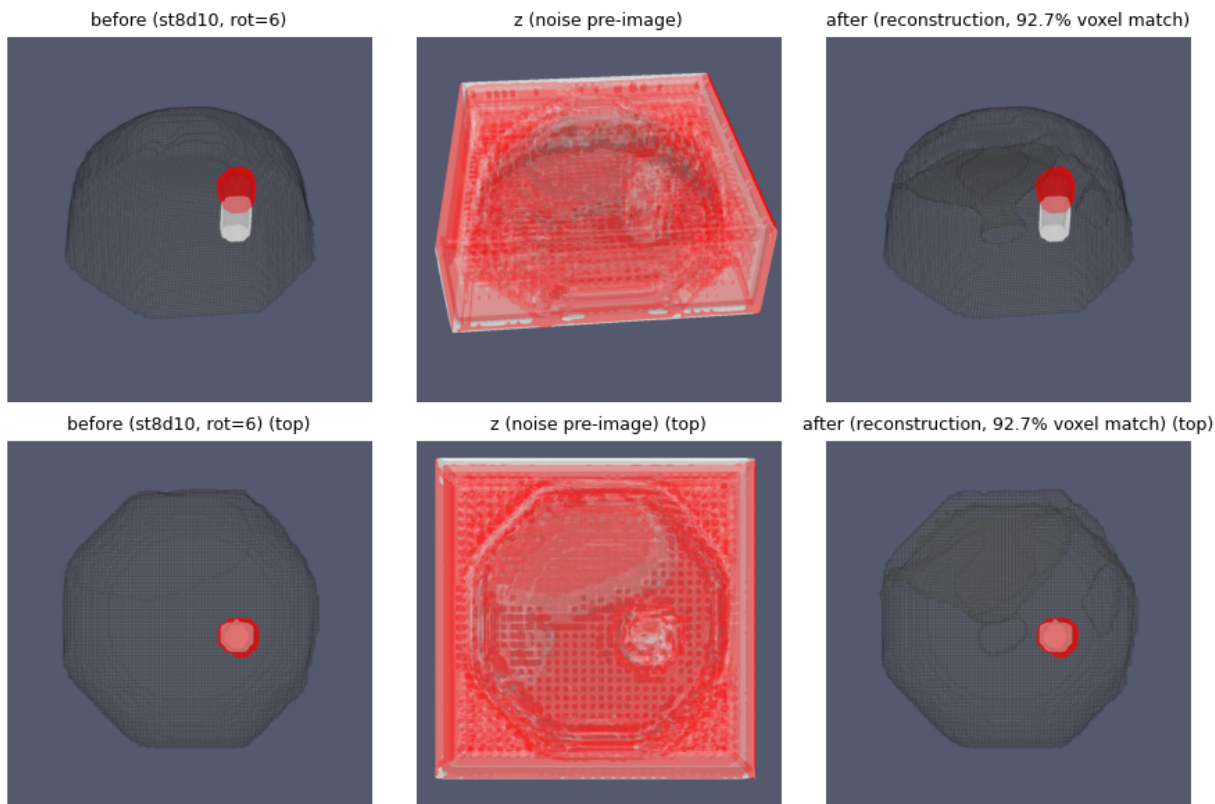


Figure 1: Before (real, rotated phantom)  $\rightarrow z$  (inverted noise pre-image)  $\rightarrow$  after (re-generated from  $z$ ). Top row: default angled view. Bottom row: top-down view. Checkpoint: 1000-epoch, class-weighted loss.

### Ruling out bad inversion

The first suspect for the broken interpolation was inversion itself. We tested this with a round-trip check (invert a real phantom, regenerate, measure voxel agreement with the original): 96–99% agreement at 50 steps, on both the original and the class-weighted checkpoints. Inversion of a single phantom is excellent either way — this rules out “inversion is unreliable” as the explanation.

## Ruling out numerical/discretization error

Next suspect: maybe 50 ODE integration steps isn’t enough resolution once the field is sharper. Rerunning the same interpolation with 150 steps instead of 50 produced essentially the same broken shape — same tear, same scattered fragments, in nearly the same positions. A genuine discretization error would be expected to improve with  $3\times$  more resolution; this didn’t, which rules out step count as the cause.

## What’s actually going on

Section 4.4–4.5 give the quantitative version of this: flow matching only ever trains on individual straight-line paths, each from one random noise sample to one specific real phantom, in a space so large ( $d \approx 1.7 \times 10^6$ ) that  $\sim 10^3$  training examples cover only a vanishing sliver of it. Nothing in training constrains what should happen at points between two different phantoms’ separate paths, and at  $t = 0$  (pure noise) that neighborhood is the least-constrained region in the entire space. Figure 2 shows a representative example: the two endpoints (blue/orange borders) are clean, and the “actual” generated lump position (green  $\times$ ) happens to land almost exactly on the naive linearly “expected” position (red star) this time — but the interpolated anatomy itself still visibly warps, growing extra bumps and a few disconnected floating specks that are not present in either endpoint. Centroid position and overall anatomical quality are two separate things the model can get wrong independently, and this example shows the latter failing even when the former happens to succeed.

We also checked 10 different real-phantom pairs at the same  $\alpha$  values directly: the failure is inconsistent rather than universal — a couple of pairs interpolated cleanly on both counts (centroid position *and* anatomy), some (like Figure 2) get the centroid right but still distort the anatomy, and others miss badly on both. That matches uneven coverage: some pairs’ inverted noise points happen to sit near a well-covered part of the space, most don’t, and it isn’t simply a function of how far apart the two phantoms’ lumps start out.

## Where things stand, and the proposed next step

### Working

Unconditional generation from noise: solid, consistent anatomy, lump present in most fixed evaluation seeds. Inversion of a single real phantom back to noise: highly accurate. The overall pipeline mechanics all run end-to-end without errors.

### Not yet working

Real-phantom interpolation via invert / slerp / forward-integrate is unreliable — it works for some pairs and produces torn, fragmented anatomy for most others. Neither more training epochs, class-weighting the loss, nor more ODE integration steps fixed this.

### Proposed next step: interpolate at partial inversion ( $t_{\text{mid}}$ )

The core idea: don’t fully invert either phantom all the way to  $t = 0$ . Only invert *partway*, to some intermediate  $t_{\text{mid}} > 0$  (e.g. 0.2–0.3), interpolate the two phantoms’ partial trajectories *there*, and integrate forward the remaining distance from  $t_{\text{mid}}$  to  $t = 1$ . Figure 3 makes this mechanical: the top row is the current (broken) approach, the bottom row is the proposal.

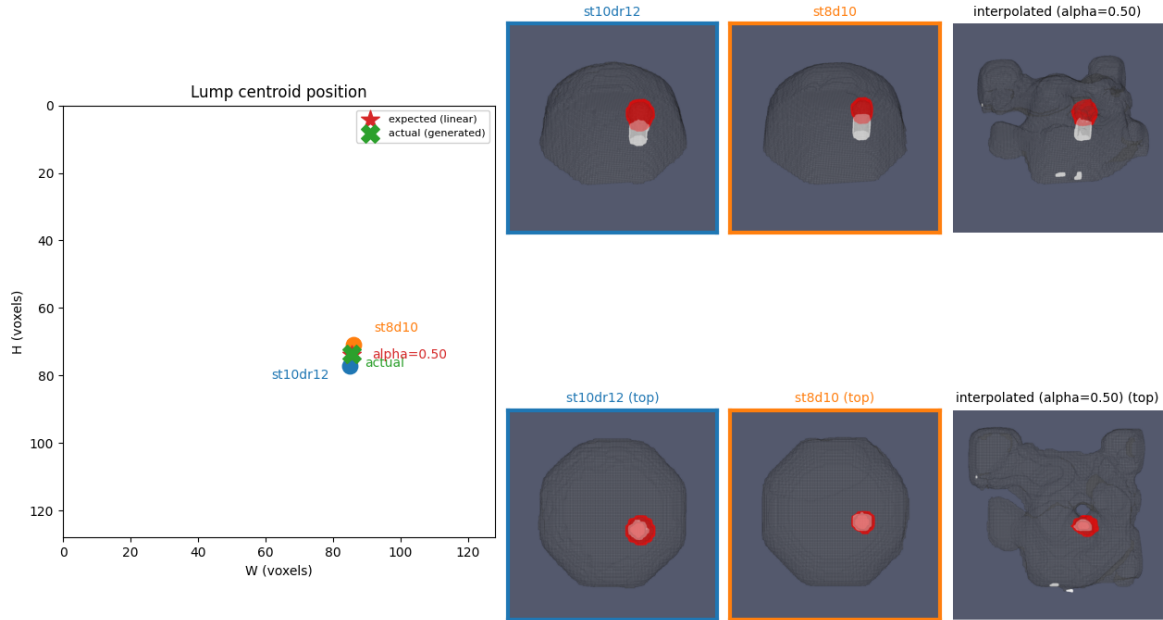


Figure 2: A representative real-phantom interpolation (pair: `st10dr12`  $\leftrightarrow$  `st8d10`,  $\alpha = 0.5$ ). Left: lump-centroid schematic, showing the naive linearly-expected position (red star) versus what the model actually generated (green  $\times$ ) — close this time. Right: renders of both endpoints and the interpolated result; note the extra bumps and small disconnected specks that appear in the interpolated anatomy but in neither endpoint.

To be explicit about the confusion this can cause: **this is not** “start from one phantom and nudge it toward the other.” Both A and B are still inverted independently and symmetrically, exactly as before — the only change is that neither inversion runs all the way to  $t = 0$ . They stop at the same intermediate time  $t_{\text{mid}}$ , get combined by the same slerp formula (Eq. (6)) at that shared time, and then the *combined* point is integrated forward the rest of the way. At  $t_{\text{mid}}$  both trajectories are already pulled somewhat toward real anatomy (Eq. (8): the conditional mean is  $t x_1$ , so larger  $t$  means closer to the real phantom), so the neighborhood between them is more likely to be a region many training trajectories actually pass near — rather than the wide-open, near-empty space at  $t = 0$  described in Section 4.5. This is the same idea behind SDEdit-style edits in diffusion models: interpolate or edit at an intermediate noise level rather than the fully-noised endpoint.

The trade-off: too high a  $t_{\text{mid}}$  leaves the ODE too little room to reshape anything, so the result collapses toward a literal voxel-wise blend of the two phantoms rather than a real interpolation; too low and the original problem returns. This needed sweeping a few values to find where it actually helps, and a small extension to `ode.py` so `invert/sample` can start/stop at an arbitrary  $t$  instead of always running the full  $[0, 1]$  range. Implemented and tested — results in Section 8.

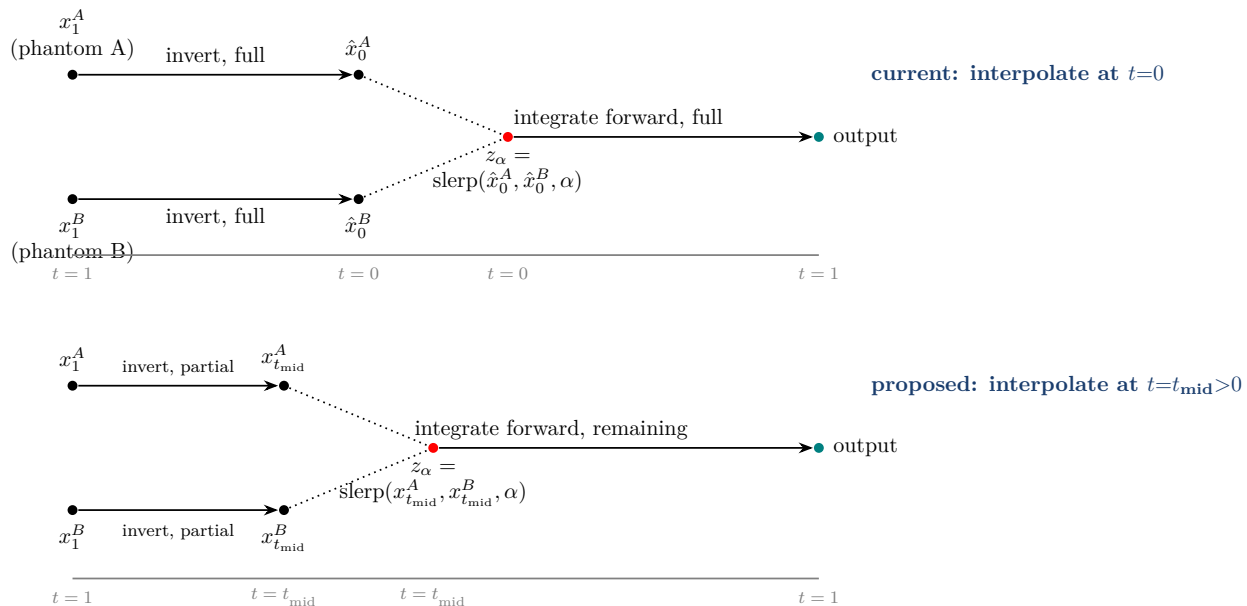


Figure 3: Both rows do the same three things — invert A, invert B, slerp, integrate forward — the only difference is *how far back* each phantom is inverted before interpolating. Top: invert all the way to pure noise ( $t=0$ ), the current (broken) approach: A and B each travel the *full* backward distance independently before ever being combined. Bottom: stop the backward integration early, at  $t_{\text{mid}}$ , so each phantom is still partway pulled toward its own real anatomy when the two trajectories are combined; only the remaining distance ( $t_{\text{mid}} \rightarrow 1$ ) is integrated forward afterward.

## The more fundamental fix: add the compression back

Section 4.4 named the real trade-off: flow matching has no bottleneck, so nothing forces the noise space to be smoothly organized as a whole.  $t_{\text{mid}}$  works *within* that constraint. The more fundamental fix is to remove the constraint: train a VAE-style encoder first, then run flow matching inside *its* compact, KL-regularized latent space instead of the raw  $d \approx 1.7 \times 10^6$ -dimensional voxel space. This is precisely the idea behind Latent Diffusion (the architecture behind Stable Diffusion): a much smaller, explicitly smoothed latent space, with flow matching layered on top of it. It is a materially bigger change than  $t_{\text{mid}}$ , but it targets the actual cause rather than working around it.

## Testing $t_{\text{mid}}$ : results

$t_{\text{mid}}$  was implemented (`flow_model/sweep_tmid.py`) and swept from 0.1 to 0.9 in steps of 0.1, on 20 real-phantom pairs  $\times$  3 alphas (60 pair-alpha combinations per  $t_{\text{mid}}$ ), using the same checkpoint as Section 6.

## Metrics, defined precisely

Two families of metric, both computed directly from the generated label volume with no rendering required.

**Centroid MAE.** For a pair  $(A, B)$  at mixing fraction  $\alpha$ , the naive linearly *expected* lump position

is

$$p_{\text{expected}}(\alpha) = (1 - \alpha)p_A + \alpha p_B, \quad (15)$$

where  $p_A, p_B \in \mathbb{R}^2$  are the real endpoints’ own lump centroids (voxel-averaged  $(w, h)$  position of the lump class). The *actual* position  $p_{\text{actual}}(\alpha)$  is the same centroid computed on the model’s generated output at that  $\alpha$ . The reported metric is the mean, over all pair-alpha combinations, of the Euclidean error between them:

$$\text{centroid MAE} = \frac{1}{N} \sum_{i=1}^N \|p_{\text{expected}}^{(i)} - p_{\text{actual}}^{(i)}\|_2. \quad (16)$$

This is exactly the green- $\times$ -to-red-star distance shown in every interpolation figure so far, just averaged into one number instead of eyeballed per figure.

**Fragmentation (connected components).** Real single-lump phantoms are, by construction, one connected body with exactly one connected lump. Let  $M_c \subset \{0, 1\}^{K \times H \times W}$  be the binary mask of voxels belonging to class-set  $c$  in the generated volume. Applying 3D connected-component labeling to  $M_c$  (26-connectivity: two voxels are connected if they share a face, edge, or corner) gives a component count  $n(c)$ . We report the mean of  $n(c)$  over all pair-alpha combinations for two choices of  $c$ : the whole foreground anatomy (insert  $\cup$  pillar  $\cup$  lump, “avg\_anatomy\_components”) and the lump class alone (“avg\_lump\_components”). A value of exactly 1 matches real data; higher values mean the generated shape has torn into disconnected pieces.

## Results

| $t_{\text{mid}}$ | centroid MAE (voxels) | avg. anatomy components | avg. lump components |
|------------------|-----------------------|-------------------------|----------------------|
| 0.1              | 10.81                 | 22.78                   | 7.18                 |
| 0.2              | 7.93                  | 25.82                   | 5.32                 |
| 0.3              | 6.86                  | 26.57                   | 2.97                 |
| 0.4              | 6.80                  | 23.50                   | 2.23                 |
| 0.5              | 6.86                  | 17.67                   | 2.62                 |
| 0.6              | 6.55                  | 7.73                    | 3.12                 |
| 0.7              | 6.40                  | 5.77                    | 5.15                 |
| 0.8              | 7.10                  | 5.07                    | 3.78                 |
| 0.9              | 7.15                  | 6.17                    | 1.72                 |

Taken at face value, this looks like partial success: centroid MAE drops by roughly 40% from  $t_{\text{mid}} = 0.1$  to its plateau around 0.6–0.7, and anatomy fragmentation drops by more than  $4\times$  over the same range. That is genuinely a large, real improvement in these two numbers.

## But: what the numbers were actually measuring

Before trusting that as “ $t_{\text{mid}}$  fixes interpolation,” we inspected the rendered examples the sweep also logged (5 example pairs per  $t_{\text{mid}}$ , both views) rather than the aggregate numbers alone — and the improvement does not mean what it first appears to. Figure 4 shows the same pair at  $t_{\text{mid}} = 0.5$ : at  $\alpha = 0.25$  the “interpolated” output is essentially *identical* to phantom A (same lump position, same pillar position) — not a 25%-blend toward B, a near-exact copy of A. At  $\alpha = 0.75$  (not shown) the same thing happens in reverse: the output is essentially a copy of B. At  $\alpha = 0.5$ , exactly at the midpoint, the output shows *both* phantoms’ lumps simultaneously, one at A’s position and one at B’s, rather than one blended lump in between. We checked this across multiple pairs

and across  $t_{\text{mid}} \in \{0.1, 0.5, 0.9\}$ : the same pattern holds everywhere in the sweep, not just at large  $t_{\text{mid}}$ .

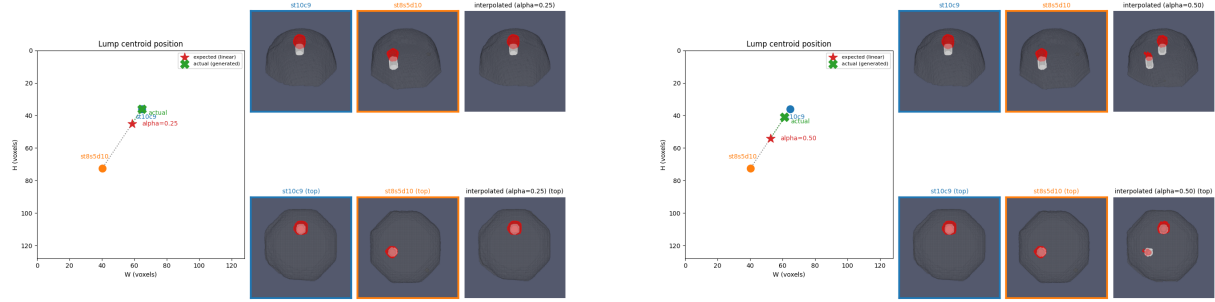


Figure 4: Same pair (`st10c9`  $\leftrightarrow$  `st8s5d10`),  $t_{\text{mid}} = 0.5$ . Left,  $\alpha = 0.25$ : the output matches phantom A almost exactly (compare lump/pillar position to the `st10c9` panel) rather than showing a 25% blend toward B. Right,  $\alpha = 0.5$ : both A’s and B’s lumps appear simultaneously (visible as two separate red regions in the top-down row) instead of one blended lump partway between them.

This means the model is not smoothly interpolating at any  $t_{\text{mid}}$  tested — it is behaving like a nearest-neighbor selector in angle: close to A’s specific trained direction it reproduces A, close to B’s it reproduces B, and exactly at the ambiguous midpoint it produces a superposition of both rather than a genuinely new, blended anatomy. This directly explains why “`avg_lump_components`” never drops cleanly to 1 anywhere in the table: at  $\alpha = 0.5$  specifically, 2 lumps (one from each endpoint) is close to the *expected* outcome of this snapping behavior, not evidence of a single well-formed interpolated lump. The centroid MAE and fragmentation improvements at higher  $t_{\text{mid}}$  are real, but they measure “looks closer to one of the two real endpoints, less torn than a full inversion would produce” rather than “genuinely interpolates between them.”  $t_{\text{mid}}$  narrows *where* the model breaks down, but does not fix the underlying issue described in Section 6 and Section 4.4: the network was only ever taught the translation *at* each individual real phantom, never in the neighborhood between two of them — and that remains true regardless of how far back either endpoint is inverted before combining them. The more fundamental fix proposed above (a compact, KL-regularized latent space) attacks that root cause directly;  $t_{\text{mid}}$ , as tested here, does not.

## Other directions worth trying next

$t_{\text{mid}}$  narrows *where* the model breaks, but Section 8 showed it doesn’t touch *why*. Four candidate next steps, roughly cheapest/least-certain to most involved/most-targeted:

### 1. Zero retraining: multi-hop interpolation through real neighbors

Instead of interpolating directly between two arbitrary real phantoms — one long hop across a space where, per Section 4.5,  $\sim 10^3$  training points cover a vanishing sliver of  $d \approx 1.7 \times 10^6$  dimensions — build a similarity graph over the real phantoms (e.g. by noise-space or lump-position distance) and, for a chosen pair  $A, B$ , find a chain of intermediate real phantoms connecting them. Interpolate each short hop instead of one long jump, so every step only ever traverses the neighborhood of two real, well-covered points. This needs no retraining at all — purely a change to which pairs get

interpolated and how many steps, testable immediately with the existing pipeline. Trade-off: with only 40 case IDs the neighbor graph is sparse, so this only helps pairs that already have plausible intermediate real phantoms between them, and does nothing for the underlying issue when two phantoms are simply far apart with nothing real in between.

## 2. New training data: synthetic in-between supervision

Nothing in training currently supervises the region between two real phantoms’ paths. Fix that directly: for pairs of real phantoms, compute an offline, geometrically-motivated “halfway anatomy” and add it as an auxiliary training target at intermediate points along that pair’s path. A standard way to build that target is per-class signed-distance-field interpolation: for two binary masks  $m_A, m_B$  of the same class, let  $\text{SDF}(m)$  be the signed distance transform (negative inside the shape, positive outside, magnitude equal to distance from the boundary). Then

$$\text{SDF}_\alpha = (1 - \alpha) \text{SDF}(m_A) + \alpha \text{SDF}(m_B), \quad m_\alpha = \mathbb{I}[\text{SDF}_\alpha \leq 0] \quad (17)$$

gives a smoothly-morphing synthetic mask at any  $\alpha$ , per class, that can be turned into a one-hot label volume and used as an extra regression target for  $v_\theta$  near the corresponding point on the  $A$ – $B$  path. This is the only option here that places real supervision in the exact spot proven broken in Sections 4.5 and 8. Trade-off: the SDF-blended target is a geometric heuristic, not verified ground truth (no real scan of a genuine intermediate phantom exists), and it requires modifying the training loop and retraining, not just a one-off inference-time experiment.

## 3. Add the compression back: a compact, learned latent space

Covered in full in Section 7: train a VAE-style encoder first, then run flow matching inside its compact, KL-regularized latent space instead of the raw  $d \approx 1.7 \times 10^6$ -dimensional voxel space (the idea behind Latent Diffusion / Stable Diffusion). This is the most direct fix for the root cause named in Section 4.4 — no bottleneck currently means nothing forces the noise space to be smoothly organized — but it is also the largest change of the four: a new model component to train, and no guarantee it converges cleanly on a dataset this small.

## 4. Smoothness regularization on the velocity field

A middle ground between (2) and (3): add a cheap penalty term during training that pushes  $v_\theta$  to respond similarly to slightly perturbed points, not only to points that lie exactly on a trained path. Concretely, for a training point  $x_t$ , sample a small perturbation  $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$  with  $\sigma$  small relative to the typical shell radius (Eq. (10)), and add

$$\mathcal{L}_{\text{smooth}} = \lambda \mathbb{E}[\|v_\theta(x_t + \epsilon, t) - v_\theta(x_t, t)\|^2] \quad (18)$$

to the training loss, with  $\lambda$  a small weight. This is a one-line addition to the training loop, no new data required. Trade-off: it only encourages smoothness in an infinitesimal neighborhood of trained points, not that the large-scale neighborhood *between* two far-apart real phantoms — which is what interpolation actually needs — becomes meaningful. Likely necessary but not sufficient on its own.

## Summary of concrete changes made this session

- Added class-weighted flow-matching loss (Eq. (13)–(14)).

- Added multi-pair, randomly-rotated real and noise interpolation with denser alpha sweeps.
- Added a top-down render alongside the default angled view for every figure.
- Color-coded each endpoint’s render border to match its point on the position schematic.
- Added the actual-vs-expected lump centroid diagnostic (Figure 2).
- Built `visualize_inversion.py` for the before  $\rightarrow z \rightarrow$  after figure (Figure 1).
- Fixed a concurrent-rendering corruption bug and a concurrent-checkpoint-overwrite bug.
- Ran round-trip and step-count diagnostics to isolate the interpolation failure’s cause.
- Worked out the dimensionality/distribution analysis in Section 4, quantitatively explaining both the linear-vs-slerp choice and the  $z$ -rendering size mismatch.
- Extended `ode.py` to support partial inversion/sampling at an arbitrary  $t$ , and built `sweep_tmid.py` to test the  $t_{\text{mid}}$  idea across  $t_{\text{mid}} \in [0.1, 0.9]$  on 20 pairs with quantitative metrics (Section 8).
- Discovered, by inspecting rendered examples rather than trusting the aggregate metrics alone, that the model snaps to whichever endpoint  $\alpha$  is closer to instead of smoothly interpolating, at every  $t_{\text{mid}}$  tested (Figure 4) — the real bottleneck is not  $t_{\text{mid}}$ , it is the lack of a compact, smoothly-organized latent space (Section 7).
- Documented four candidate next directions, from a zero-retraining experiment to training-data and architecture changes (Section 9).

## Part II: pivoting away from interpolation — the 180-degree rotation task

Section 9’s four candidate fixes all assume the underlying interpolation idea is sound and just needs a smoother latent space, a denser training signal, or a more forgiving target. The next work session took a different approach entirely: rather than continuing to chase the  $A \rightarrow B$  interpolation problem, pivot to a self-supervised task with the one property the interpolation task fundamentally lacked — *exact ground truth* for every prediction, so progress could finally be measured against a known-correct answer instead of only judged by eye.

### 10. The new task: learn to rotate a phantom 180 degrees

Every phantom in the dataset is augmented with  $8 \times$  in-plane rotation at  $45^\circ$  increments (orientation  $o \in \{0, \dots, 7\}$ , Section 1). Because  $180^\circ = 4 \times 45^\circ$ , orientation  $o$  and orientation  $(o + 4) \bmod 8$  of the *same* underlying phantom are an exact, bit-for-bit  $180^\circ$  pair — confirmed lossless, since `torchvision`’s nearest-neighbor rotation at exactly  $180^\circ$  reduces to a pure array flip with no interpolation artifacts. This gives a new task with a property interpolation never had: pick any phantom at any orientation as  $x_0$ , and its exact target  $x_1$  is known in closed form, with no scan, no heuristic, and no synthetic blending needed.

$$x_0 = \text{rotate}(x_{\text{raw}}, o), \quad x_1 = \text{rotate}(x_{\text{raw}}, (o + 4) \bmod 8). \quad (19)$$

Splitting phantoms by whether the lump sits right or left of the volume’s vertical center line ( $w \geq 63.5$ , “set A” vs “set B”) confirmed the framing geometrically: across all 984 augmented instances, the split came out exactly balanced (492/492) with zero ambiguous cases, and by construction every  $(o, o+4)$  pair lands on opposite sides — so training bidirectionally on all 8 orientations as  $x_0$  automatically covers both  $A \rightarrow B$  and  $B \rightarrow A$  without any explicit filtering.

Because  $x_0$  is now always real data (never Gaussian noise), generation no longer needs the inversion step at all: sampling is a single forward integration of Eq. (4) starting from the real  $x_0$ . This sidesteps the entire interpolation failure mode from Part I by construction — there is no noise-space midpoint to be inaccurate about.

### 11. First result: loss goes down, nothing moves

Training with the unmodified loss from Eq. (3) (now with  $x_0$  real instead of noise) produced a striking failure: `train/val_loss` dropped substantially over hundreds of epochs, yet the *decoded* prediction was — voxel for voxel, checked programmatically, not just by eye — **100% identical to**  $x_0$  on every one of 10 tested validation pairs, unaffected by using up to  $20 \times$  more ODE integration steps at inference. The resolution came from separating two different questions: is the raw, continuous velocity prediction directionally correct (measured *on* the exact training line, i.e. with an oracle view of  $x_1$ ), versus does the actual multi-step, self-directed generation process reach the target. A broader check (20 validation pairs, several  $t$  values,  $t = 0.5$  excluded as the one point where the one-hot blend is maximally ambiguous regardless of training quality) showed the network already had a strong, consistent *directional* signal ( $\cos\text{-sim} \approx 0.65$ ) at that point in training — the problem was pure *magnitude*: predicted velocity was only  $\sim 20\%$  of the size needed to actually flip a voxel’s class after integration, a classic symptom of plain MSE regression hedging toward an undercommitted answer once loss is already fairly low.

## 12. Fix 1: a cross-entropy term on the endpoint estimate

Since  $v^* = x_1 - x_0$  is constant along the line, algebra gives an “endpoint estimate” from any point on the trajectory using only quantities already computed in the forward pass:

$$\hat{x}_1 = x_t + (1 - t) v_\theta(x_t, t), \quad (20)$$

exact when  $v_\theta = v^*$ . A first attempt supervised this with an *MSE* loss against the true  $x_1$  — but algebraically,  $(\hat{x}_1 - x_1)^2 = (1 - t)^2(v_\theta - v^*)^2$ : exactly  $(1 - t)^2$  times the existing velocity loss, same minimizer, same gradient direction, not a new training signal at all (caught before wasting a training run on it). The actual fix uses **cross-entropy** instead:

$$\mathcal{L}_{\text{pixel}} = \text{CE}(\hat{x}_1, \text{argmax}(x_1)), \quad \mathcal{L} = \mathcal{L}_{\text{velocity}} + \lambda_{\text{pixel}} \mathcal{L}_{\text{pixel}}. \quad (21)$$

Unlike MSE, cross-entropy’s gradient does *not* shrink as a prediction approaches “good enough” — it stays large as long as the wrong class is still winning at a voxel, directly countering the undercommitment found in Section 11. This worked: the decoded prediction started moving for the first time, with real, substantial repositioning (roughly a 50–60% reduction in lump-centroid distance versus doing nothing) within the first  $\sim 150$  epochs of a fresh 1000-epoch run.

## 13. The centroid metric lies: a new failure mode

Direct visual inspection of the trained predictions (not just their summary statistics) told a different story than the centroid-distance numbers: predictions were a **massive, fragmented crescent-shaped blob** covering much of the volume’s cross-section, not a small relocated lump — confirmed quantitatively as  **$\sim 24$  disconnected predicted lump components versus  $\sim 1$  expected** on real phantoms. Centroid distance and overall voxel-agreement both stayed misleadingly reasonable throughout, because a large blob’s center of mass can coincidentally land near the true position, and because background voxels dominate voxel-agreement regardless. The root cause is a specific property of PyTorch’s class-weighted cross-entropy: the **weight** argument scales the loss by each voxel’s *true* class, not its predicted one, so missing a real lump voxel costs up to  $12.57\times$  but wrongly predicting lump at a true-background voxel costs only  $1.0\times$  — a direct, asymmetric incentive to over-predict the rare classes wherever the network is unsure.

## 14. Fix 2: symmetric volume regularization, a decaying schedule, and EMA

Three complementary changes addressed this:

- **Soft Dice loss** on the lump/pillar channels, added alongside Eq. (21):

$$\mathcal{L}_{\text{dice}} = 1 - \frac{2 \sum_v p_v t_v + 1}{\sum_v p_v + \sum_v t_v + 1}, \quad \mathcal{L} += \lambda_{\text{dice}} \mathcal{L}_{\text{dice}}. \quad (22)$$

Unlike cross-entropy, Dice is symmetric in false positives and false negatives — predicting excess volume directly grows the denominator without growing the overlap numerator, so over-prediction is penalized even where CE does not penalize it.

- **Decaying  $\lambda_{\text{pixel}}$** : linearly interpolated from 1.0 down to 0.3 over training (by absolute epoch, so it continues correctly across a resumed run) — cross-entropy’s magnitude push has no natural ceiling and was found to sustain a  $\sim 1.25\text{--}1.3\times$  magnitude overshoot for hundreds of epochs when held constant; fading it out lets the bounded, well-behaved velocity MSE and Dice terms dominate refinement later.

- **EMA of the model weights** (decay 0.999,  $\sim 10$ -epoch smoothing window at this task’s step rate), used for every preview, diagnostic, and checkpoint save instead of the raw training weights — motivated directly by the aggregate lump-centroid-error metric swinging between  $\sim 17$  and  $\sim 60$  voxels from one 25-epoch checkpoint to the next with no clear trend, a classic symptom of noisy per-step weights in diffusion/flow-matching training.

This combination fixed the blob: predicted lump component counts fell from the 20–60 range to mostly 2–14, much closer to the  $\sim 1$  expected.

## 15. A properly trusted metric: mask overlap, not centroid distance

Even after fix 14, a closer look showed the improvement was still partly illusory. Directly computing lump-mask **IoU** and **Dice** (intersection-over-union and Dice coefficient between the predicted and true *binary* lump mask, not just their centroids) gave a mean IoU of only  $\approx 0.07$  and Dice  $\approx 0.13$  across ten validation pairs — versus an exact 0.000 for a trivial “never move anything” baseline (meaningful, since the two positions are always  $180^\circ$  apart and can never overlap by chance), but far from good. Table 1 makes the point concretely: a  $5 \times 5$  predicted blob containing one true voxel scores  $\text{IoU} = 1/25 = 0.04$  while its centroid can be dead-on. Mask IoU/Dice (per class and combined lump+pillar) is now the trusted metric throughout the codebase; centroid distance is kept only as a secondary, easily-misled signal.

| Scenario                                  | Centroid error | IoU               | Dice              |
|-------------------------------------------|----------------|-------------------|-------------------|
| Never move (trivial baseline)             | large          | 0.00              | 0.00              |
| Small blob near true position, poor shape | can look small | 0.04–0.10         | 0.08–0.18         |
| Correct, precisely-shaped prediction      | $\approx 0$    | $\rightarrow 1.0$ | $\rightarrow 1.0$ |

Table 1: Why mask overlap and centroid distance can disagree sharply on the same prediction.

## 16. Ruling out numerical error: it’s exposure bias, not integration accuracy

Before touching the loss again, the live checkpoint was tested for a simpler possible cause: insufficient integration accuracy. Switching the sampler from Euler to Heun’s method (second-order predictor–corrector, already implemented) and separately tripling the step count ( $30 \rightarrow 100$ ) both left mean combined IoU essentially unchanged ( $0.094 \rightarrow 0.094 \rightarrow 0.090$ ). This rules out numerical discretization error and points squarely at *exposure bias*: the network was only ever trained at points exactly on the straight line between a real  $(x_0, x_1)$  pair (Eq. (1)), so it can fit that narrow, teacher-forced objective well — correct direction, roughly correct magnitude — while still being inaccurate the moment its own multi-step, self-directed trajectory departs from that line, which it does almost immediately (measured drift from the true line grows roughly linearly step over step, reaching a magnitude comparable to the pair’s own  $x_0$ -to- $x_1$  distance well before the trajectory completes).

## 17. Fix 3: training on the model’s own rollout, not just the teacher-forced line

The direct fix is to expose the network, during training, to states reached by *its own* imperfect steps, not only the idealized line. A short, gradient-tracked rollout from  $x_0$  using the model’s own predictions at each step,

$$x^{(0)} = x_0, \quad x^{(i+1)} = x^{(i)} + v_\theta(x^{(i)}, i \Delta t) \Delta t, \quad i = 0, \dots, k-1, \quad \Delta t = t_{\text{end}}/k, \quad (23)$$

followed by the same endpoint-estimate trick (Eq. (20)) and the same cross-entropy + Dice terms (Eqs. 21, 22) evaluated at the rolled-out state instead of a point on the true line, directly penalizes drift instead of only ever rewarding accuracy on a path the network will never exactly follow at inference. Unlike Eq. (1), every step of Eq. (23) feeds forward through the network with gradients retained, so this costs  $k$  forward/backward passes per training step instead of one and needs the full computation graph kept live across all  $k$  steps.

This loss proved unstable when applied to a freshly-initialized network: with  $k=3$ ,  $t_{\text{end}}=0.3$ , applied every other batch, predicted lump components on a from-scratch run *grew* across the first three epochs (754  $\rightarrow$  1794  $\rightarrow$  13370) rather than settling down — plausibly because compounding a still-near-random field across  $k$  real self-steps early in training produces wildly out-of-distribution states, and the resulting large, noisy gradients destabilize the network before it has any baseline competence to build on. Applying the identical loss as a *continuation* of an already-trained checkpoint (resumed from epoch 469 of the fix-14 run) showed no such instability — component counts stayed low and stable across five additional epochs. An extended run applying this as a fine-tuning phase on top of the fix-14 checkpoint was launched to test whether mask IoU/Dice actually trends upward over a longer horizon; results were not yet available at the time of writing.

## 18. Supporting infrastructure built alongside the modeling work

- **RotationPairDataset**: bidirectional  $(x_0, x_1)$  pairing via Eq. (19) for both train and validation splits (orientation is the task variable here, not an augmentation nuisance to be dropped from validation).
- Checkpoint resume support (`--resume_from`, plus reattaching to the same wandb run id) added specifically so a promising run could be extended rather than restarted from scratch — used repeatedly once the rollout-loss fine-tuning strategy above was adopted.
- DataLoader `num_workers` support: the paired dataset’s `__getitem__` does two rotations per item (versus the unconditional task’s free noise draw), which was starving the GPU at `num_workers=0`; parallelizing gave a measured  $3.4\times$  speedup (127s/epoch  $\rightarrow$  37s/epoch) with identical loss values, confirming it was a pure I/O fix.
- A 50-pair aggregate diagnostic pass, run periodically during training and logged as a proper trend graph, replacing an earlier design that logged one scalar per individual preview pair (too noisy pair-to-pair to read a trend off).
- `eval_rotate180.py`: compares any set of checkpoints (e.g. the 100/250/500/1000 epoch milestones) on the full held-out validation set with exact ground truth, auto-detecting each checkpoint’s architecture from its own saved config rather than requiring hand-matched CLI flags.

## Summary of concrete changes made this session (Part II)

- Pivoted from noise-space interpolation to a self-supervised 180°-rotation task with exact, closed-form ground truth for every prediction (Section 10).
- Built and verified the set A/B lump-position split (492/492 balanced, 0 ambiguous) and a showcase figure.
- Diagnosed, by checking the decoded output directly rather than trusting the loss curve, that plain velocity MSE never moved the prediction at all; traced this to an undercommitted velocity magnitude, not a directional error (Section 11).
- Added and then *rejected* a redundant MSE-based pixel loss after catching, by algebra, that

it was an exact reweighting of the existing velocity loss; replaced it with a genuinely new cross-entropy signal (Section 12).

- Found, via direct visual inspection, that the cross-entropy fix produced a massive fragmented blob ( $\sim 24$  components vs  $\sim 1$  expected) that the centroid/voxel-agreement metrics had been hiding; traced this to cross-entropy’s asymmetric true-class-only weighting (Section 13).
- Added Dice loss, a decaying pixel-loss schedule, and EMA weights to fix the blob and stabilize training (Section 14).
- Added mask IoU/Dice (lump, pillar, combined) as the trusted evaluation metric, demonstrating concretely why centroid distance is not sufficient on its own (Section 15).
- Ruled out numerical integration error as the remaining bottleneck (Heun’s method and  $3\times$  more steps: no change) and confirmed exposure bias as the root cause (Section 16).
- Implemented a gradient-tracked rollout-consistency loss that trains directly on the model’s own multi-step behavior; found it unstable from a fresh initialization but stable as a fine-tuning phase on an already-trained checkpoint, and launched an extended run to evaluate it (Section 17).
- Built checkpoint-resume infrastructure, a  $3.4\times$  DataLoader speedup, an aggregate diagnostic pass, and a cross-checkpoint evaluation script (Section 18).

## Part III — Cleanup, a rendering bug, and the rollout-loss follow-through

### 19. Removing the legacy metrics everywhere, not just from the headline number

Mask IoU/Dice had been adopted as the trusted metric (Section 15), but the older voxel-agreement and centroid-error numbers were still being computed and logged alongside it in the training-time diagnostic pass and in wandb, which made the dashboards noisy and easy to misread at a glance. `metrics.rotation_metrics` keeps those legacy fields internally (cheap to compute, occasionally useful for debugging a specific pair by hand) but every logging site was audited and cut down to only the six trusted numbers: `lump_iou`, `lump_dice`, `pillar_iou`, `pillar_dice`, `combined_iou`, `combined_dice`. This touched the training loop’s diagnostic pass, the qualitative figure’s annotation text box, and both evaluation scripts.

### 20. A machine-wide GPU memory leak in the rendering path

Shortly after Section 19’s cleanup, the qualitative wandb figures went black across every run. The obvious suspect — two training processes sharing GPU memory — turned out to be wrong: `nvidia-smi` showed one training process holding 23.8 GB as a *graphics* context allocation on a GPU that process never used for its actual PyTorch tensors, i.e. the leak was in off-screen rendering, not in PyTorch. The renderer (`PyVista`, wrapping VTK) never closed its `Plotter` objects on the off-screen path, so every rendered figure permanently pinned a VTK graphics context until the GPU’s memory was exhausted and subsequent renders silently came back black. Confirmed via `git stash/git stash pop` that this bug pre-existed this session’s own changes (it was a latent bug, not a regression introduced by the metrics cleanup), reproduced it in isolation, then fixed it with an explicit `plotter.close()` in the renderer’s `finally` block. Verified by killing both affected processes, confirming the GPU’s memory dropped from 23.8 GB to 288 MB, and re-rendering a real phantom to confirm correct (non-black) output.

## 21. Extending and repeating the rollout-loss run

Section 17 left an open question — does the rollout-consistency fine-tune actually trend upward over a longer horizon? Three further training decisions followed directly from watching that run and wanting a clean answer:

- The fine-tune was restarted under a *fresh* wandb run id rather than continuing to log into the pre-cleanup run, so the mask-only dashboard (Section 19) and the rendering fix (Section 20) would both be visible from the start of a clean run rather than appearing mid-way through an existing one’s history.
- A *second*, fully independent run was launched from scratch using only the rollout-consistency loss (no fix-14 warm start), to separate two possible explanations for the fine-tune’s behavior: is rollout training itself weak, or is it constrained by starting from a checkpoint already biased by the earlier losses?
- After the fine-tune run showed slow but real improvement, it was extended by a further 500 epochs under the same wandb run id (rather than starting yet another run), while the from-scratch run continued independently on a separate GPU. Both were run concurrently once GPU headroom allowed it, deliberately waiting for a GPU to free up naturally rather than reducing batch size to force a fit on a contended GPU.

## 22. IoU versus Dice: why report both

Both are set-overlap scores between the predicted and true binary mask,  $A$  = predicted voxels,  $B$  = true voxels:

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|}, \quad \text{Dice} = \frac{2|A \cap B|}{|A| + |B|}.$$

They are monotonic transforms of each other for a *single* pair ( $\text{Dice} = 2\text{IoU}/(1 + \text{IoU})$ , so  $\text{Dice} \geq \text{IoU}$  always, with the gap largest at moderate overlap and vanishing at  $\text{IoU} \rightarrow 0$  or  $1$ ), but that exact relationship does not carry over to *means* across many pairs, since the transform is nonlinear — averaging first and transforming, versus transforming first and averaging, give different numbers. Both are reported per-class (lump, pillar) and combined so a reader can sanity-check one against the other rather than trusting a single scalar.

## 23. eval\_trajectory.py: full-dataset metrics plus a trajectory figure

A new script combines two things the existing `eval_rotate180.py` did not do together: a full-validation-set (and optionally a matched train-sample) metrics pass on one checkpoint, and a qualitative figure showing how a single prediction evolves across the ODE integration from  $t=0$  to  $t=1$ , not just its start and end points. `ode.sample` was extended with an opt-in `return_trajectory` flag (default off, verified to leave every existing caller’s behavior unchanged) that returns the state after every integration step instead of only the final one.

The trajectory figure went through two rounds of redesign based on direct feedback on how useful the first drafts actually were. The first version paired a per-frame image filmstrip with a line chart of Dice score over the trajectory; this was replaced with a schematic plotting the lump centroid’s  $(W, H)$  position at every  $t$  (color-graded by  $t$ , true  $180^\circ$  target marked separately), since the position-over-time view is more directly diagnostic of *whether the model moves the lump at all* than a Dice-vs- $t$  curve is. The second round rebalanced panel sizes so the frame-by-frame filmstrip — the main qualitative evidence — dominates the figure, with the original/predicted panels shown small alongside it rather than each claiming half the figure’s width.

## Summary of concrete changes made this session (Part III)

- Audited every logging site (training diagnostics, qualitative figure annotations, both eval scripts) to remove the legacy voxel-agreement/centroid-error metrics, keeping only the six trusted mask IoU/Dice numbers (Section 19).
- Diagnosed and fixed a machine-wide VTK/PyVista GPU memory leak in the off-screen rendering path that was silently corrupting every logged figure; verified the fix predates and is independent of this session’s other changes (Section 20).
- Restarted the rollout fine-tune under a fresh wandb run id, launched an independent from-scratch rollout-only run, and extended the fine-tune by a further 500 epochs once it showed real improvement, running both concurrently once GPU headroom allowed it (Section 21).
- Built `eval_trajectory.py`: a full-dataset metrics pass plus a per-example trajectory figure showing the ODE integration state at every step, iterated twice on figure design based on direct feedback (Section 23).

## Part IV — Choosing and validating the final checkpoint

### 24. A further layout fix, and a scare that turned out to be a stale checkpoint

The trajectory figure went through one more redesign round: nesting the top (original/ predicted) and bottom (filmstrip) rows as two independent grids, each filling its own full width, removed a leftover unused-column gap the earlier version left next to the predicted panel (`tight_layout` does not compact nested grids correctly; switching to matplotlib’s `constrained_layout` fixed it). While iterating on this, a full 176-pair evaluation of the rollout fine-tune’s `latest.pt` (raw, non-EMA weights, epoch  $\approx 1442$ ) came back with combined Dice  $\approx 0.19$  — markedly worse than the  $\approx 0.45$  this same run’s own training-time diagnostic had reported shortly before. This looked like a real regression at first. It was not: the training-time diagnostic evaluates *EMA* weights (Section 14), while the ad-hoc full evaluation had been pointed at the *raw* checkpoint — and, separately, the raw weights kept changing checkpoint-to-checkpoint (epoch 1442  $\rightarrow$  1464 across successive re-renders) as the run continued past its best point. Both effects compounded to produce a misleadingly bad number, not a genuine collapse in the model’s ability.

### 25. Selecting the best checkpoint: raw vs. EMA, epoch 1250 vs. 1500

To settle this properly, every checkpoint actually saved by the fine-tune run was scored under identical conditions (same 50 held-out val pairs, same integration settings): Two things stand

| Checkpoint              | lump Dice    | pillar Dice  | combined Dice | combined IoU |
|-------------------------|--------------|--------------|---------------|--------------|
| epoch 1250 (raw)        | 0.451        | 0.153        | 0.422         | —            |
| <b>epoch 1250 (EMA)</b> | <b>0.498</b> | <b>0.268</b> | <b>0.478</b>  | <b>0.344</b> |
| epoch 1500 (raw)        | 0.470        | 0.262        | 0.448         | —            |
| epoch 1500 (EMA)        | 0.381        | 0.120        | 0.354         | —            |

Table 2: Every checkpoint saved by the rollout fine-tune run, scored on the same 50 val pairs. Epoch 1250’s EMA weights are the clear winner on every metric.

out. First, EMA beats raw weights at epoch 1250 but *loses* to raw weights at epoch 1500 — the smoothing that helps mid-run becomes a liability once the underlying weights have drifted past

their best point, since EMA is then averaging good history against a worse recent trend. Second, and more importantly: **training for longer did not help**. The extra 250 epochs (1250  $\rightarrow$  1500) requested to see whether the fine-tune “was slowly learning” instead made every metric worse for the EMA weights, and only partially recovered it for the raw weights. Epoch 1250 EMA is the best model this run produced, not the final one.

## 26. Final validated result

Running the full evaluation script on epoch 1250 (EMA) — all 176 held-out validation pairs, plus a matched sample of 176 training pairs to check for overfitting — gives Table 3.

| Split                  | lump Dice (mean) | pillar Dice (mean) | combined Dice (mean $\pm$ std) |
|------------------------|------------------|--------------------|--------------------------------|
| Validation (176 pairs) | 0.472            | 0.267              | $0.453 \pm 0.151$              |
| Train (176 pairs)      | 0.429            | 0.247              | $0.405 \pm 0.203$              |

Table 3: Final checkpoint, full-dataset evaluation. Validation Dice is not lower than train Dice (gap  $-0.048$ ), ruling out overfitting as an explanation for the result.

This is, by a wide margin, the best result this task has produced: combined Dice more than doubled the  $\approx 0.19$  figure that briefly looked like the state of the art (Section 24), and roughly  $6\times$  the very first post-blob-fix result of  $\approx 0.07$  (Section 15). Figure 5 shows the full  $t=0 \rightarrow 1$  trajectory for two held-out validation pairs: the lump visibly separates from its starting position early in the integration and settles close to its true  $180^\circ$  target, with a clean, compact predicted shape rather than the diffuse multi-component blob of earlier attempts (Section 13) or the oversized mass seen at later, overtrained checkpoints (Section 24).

## Summary of concrete changes made this session (Part IV)

- Fixed a residual layout bug in the trajectory figure (unused grid columns leaving a blank gap) by giving the top and bottom rows independent nested grids under `constrained_layout` (Section 24).
- Diagnosed an apparent Dice regression as an evaluation artifact — raw vs. EMA weights and a checkpoint that kept changing between renders — rather than a genuine model collapse (Section 24).
- Scored every checkpoint the rollout fine-tune run actually saved (raw and EMA, at epochs 1250 and 1500) under identical conditions and found the extra 250 epochs of training made the result *worse*, not better, on both weight variants (Section 25).
- Selected epoch 1250 (EMA) as the final model and validated it on the full 176-pair validation set plus a matched train sample, confirming combined Dice  $0.453 \pm 0.151$  with no overfitting (Section 26).

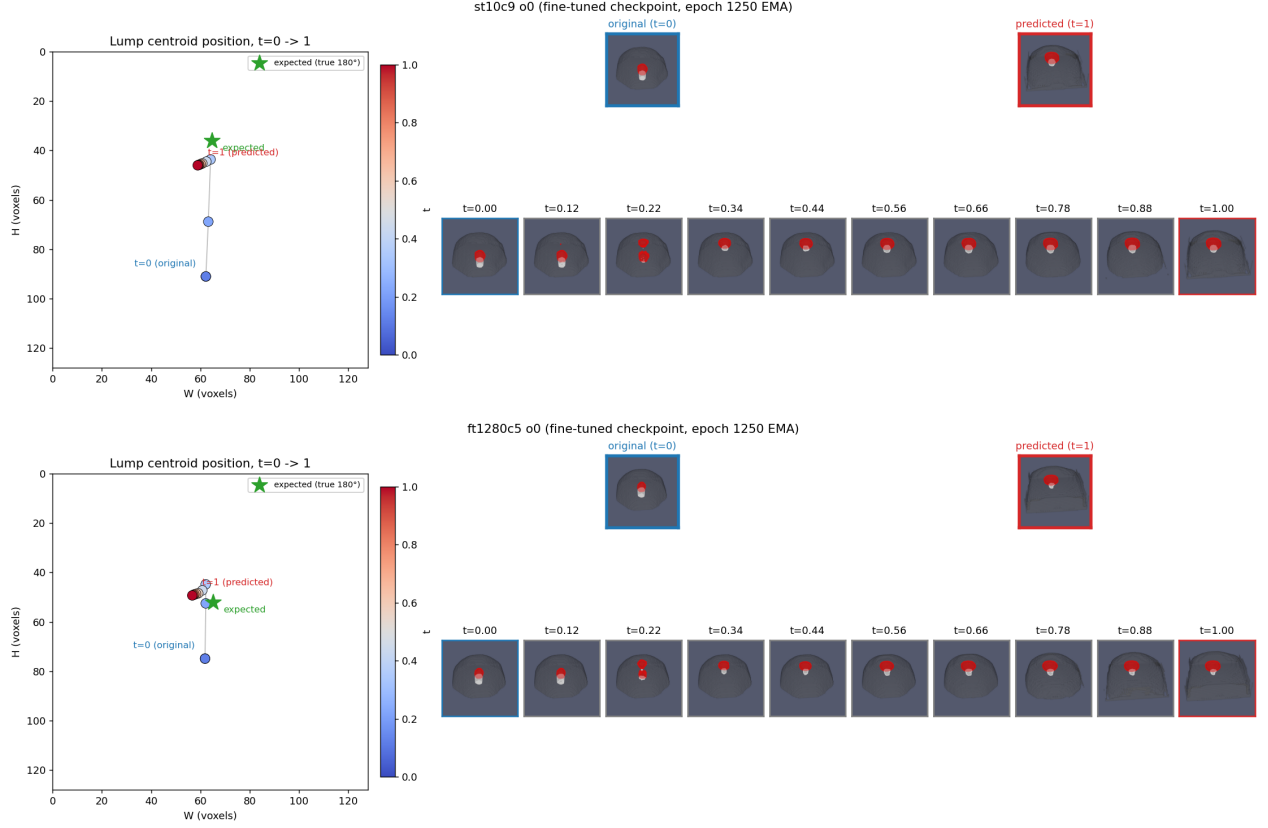


Figure 5: Full  $t=0 \rightarrow 1$  trajectory for the final checkpoint (epoch 1250, EMA weights) on two held-out validation pairs. Left: the lump centroid’s path through the volume, color-graded by  $t$ , against the true  $180^\circ$  target (green star). Right: original phantom, every intermediate integration step, and the final prediction.